

Main Critical Errors

1. 404 Error When Joining

Observed:

Users hit a 404 when trying to enter the event and had to restart/login again.

Problem:

The frontend routing or event session state is breaking.

Likely causes:

- Invalid event session token
- Race condition during join
- Frontend route exists before backend state exists
- Event room initialization delay

Fix:

- Add retry/reconnect logic
- Never expose raw 404 to user
- If session fails → automatically revalidate event state
- Event join should be idempotent

Expected behavior:

User presses Join → always ends in one of:

- Joined successfully
- Waiting room
- Reconnecting

Never a dead route.

2. User Exists But Cannot Be Matched

Observed:

Stefan Avivson account was counted in participant count but never entered matchmaking.

This is extremely important.

Problem:

Participant state != matchmaking eligibility state

Meaning:

System says:

“User exists”

But matchmaking says:

“User unavailable”

That means your state machine is fragmented.

Current broken logic:

Participant count = 7

Matchmaking count = 6

That should never happen.

Fix:

Create strict participant states:

- REGISTERED
- JOINED_EVENT
- READY_FOR_MATCH
- MATCHED
- IN_ROOM
- LEFT_ROOM
- DISCONNECTED
- HOST
- CO_HOST

Only READY_FOR_MATCH users enter algorithm.

Currently you likely have ghost users.

3. Unmatched Users Cannot Be Reassigned

Observed:

Host could see unmatched participant but could not manually place them into room.

Problem:

No manual override tools.

This is fatal for live event operations.

Fix required:

Host dashboard must allow:

- Drag user into room
- Assign user to room
- Create emergency room
- Merge rooms
- Remove from room
- Force rematch

Without this the host loses control of the event.

4. Frontend And Backend Not Syncing

Observed:

Round ended but UI still showed active round.

This is one of the most dangerous issues.

Problem:

Frontend optimistic state != backend actual state

Meaning:

Frontend says:

“Round active”

Backend says:

“Round ended”

This creates cascading errors:

- Wrong matchmaking
- Duplicate rooms
- Stuck timers
- Wrong statistics
- Ghost participants

Fix:

Frontend must NEVER assume event state.

Frontend should:

- Poll backend

or

- Use websocket authoritative state

Backend must be source of truth.

5. Mutual Match Statistics Wrong

Observed:

System displayed:

“3 mutual matches”

But visual/statistical output only showed 2.

Problem:

Analytics aggregation inconsistency.

Possible causes:

- One-way match counted as mutual
- Delayed feedback write
- Duplicate filtering issue
- Frontend cache issue

Fix:

Define mutual match explicitly:

Mutual match =

User A says YES

AND

User B says YES

Nothing else counts.

Then:

- Store centrally
- Recalculate server-side
- Render only backend result

6. Feedback Has No Admin Visibility

Observed:

No central admin visibility for ratings/feedback.

Problem:

Data exists but is operationally useless.

Fix:

Admin analytics dashboard required.

Must include:

- Event feedback overview
- User satisfaction
- Match success rates
- Re-match interest
- Dropoff rate
- Most liked users
- Event quality score
- Connection graph

Otherwise you cannot improve matching.

7. Co-host System Broken / Undefined

Observed:

Nobody knew how to assign co-host properly.

Problem:

Role architecture incomplete.

Current system likely only supports:

- USER
- HOST

But co-host is treated partially as user and partially as host.

Fix:

Explicit roles:

- USER
- HOST
- CO_HOST
- ADMIN

Each with permissions.

Co-host permissions:

- Match users
- Start round
- End round
- Manage rooms
- View analytics

But:

- Should NOT enter matchmaking
- Should NOT enter breakout rooms automatically

8. Hosts And Co-hosts Enter Breakout Rooms

Observed:

Host and co-host were automatically matched into rooms.

This is incorrect behavior.

Problem:

Operational roles treated as participants.

Fix:

Hosts/co-hosts must have:

EXCLUDE_FROM_MATCHMAKING = TRUE

Unless explicitly toggled:

“Participate as attendee”

9. Room Creation Logic Broken

Observed:

Creating manual room caused additional problems and orphaned participants.

Problem:

Room lifecycle not synchronized with matchmaking lifecycle.

Likely:

- Room exists without participant mapping

- Participant exists without room ownership
- Room capacity logic broken

Fix:

Every room must have:

- room_id
- round_id
- participant_ids
- state
- timer
- host/controller

And all room operations must be transactional.

10. UX Problem: Double Confirmation

Observed:

“You have to press it twice.”

That is pure UX failure.

Problem:

Action flow unclear.

Fix:

One action = one result.

Never:

- Match → Start → Confirm → Activate ambiguously

Instead:

1. Generate matches
2. Review
3. Start round

Very explicit.

11. Event State Visibility Is Weak

Observed:

Host cannot clearly see:

- unmatched users
- room state
- disconnected users
- active/inactive participants

Fix:

Host control center required.

Should display:

- Total participants
- Ready users
- Matched users
- Unmatched users
- Disconnected users
- Room occupancy
- Timer status
- Feedback completion

12. Naming/Identity Confusion

Observed:

“Chief developer” title confusion.

Multiple Stefan accounts.

Personal/work account confusion.

Problem:

Identity layer weak.

Fix:

- Clear profile identity
- Clear account type
- Clear role label
- Duplicate account detection
- Event nickname handling

Strategic Problem Underneath All Of This

The real issue is this:

RSN currently behaves like:

“UI screens connected together”

But it needs to behave like:

“A real-time multiplayer event operating system.”

That changes everything.

Because now you are managing:

- concurrency
- live state synchronization
- authority
- matchmaking integrity
- role hierarchy
- room orchestration
- event recovery
- participant recovery

This is closer to:

- Zoom breakout rooms
 - multiplayer game lobbies
 - Discord voice orchestration
- than a normal web app.

That means:

The backend state machine must become extremely strict.

Right now the system appears forgiving.

Forgiving systems become chaotic systems in live events.

You need:

- authoritative backend
- strict state transitions
- role segregation
- event orchestration layer
- recovery logic
- host override controls
- live synchronization architecture

Otherwise scaling RSN events will become operationally unstable very fast.